LAB ASSIGNMENT REPORT

DAY 33

Course: Web Security & Penetration Testing | Assignment: DAY33-MFA-SESSION

Jiwosh Shrestha

MFA FUNDAMENTALS, PASSWORD & SESSION SECURITY

(18 MARKS)

1. What is Multi-Factor Authentication? Give one example using two different authentication factors. Multi-Factor Authentication (MFA) is a

security mechanism that requires a user to present two or more independent authentication factors from different categories before access is

granted. The three classic categories are: knowledge (something you know), possession (something you have), and inherence (something you

are). Example: A user logs in with a password (knowledge factor) and then enters a one-time code received on their registered mobile device

(possession factor).

2. Explain the relationship between 2FA and MFA. Two-Factor Authentication (2FA) is a specific subset of Multi-Factor Authentication that

requires exactly two distinct factors. MFA is the broader umbrella term that covers any authentication scheme using two or more factors. In

practice the terms are often used interchangeably, but technically all 2FA is MFA, while MFA can also include three or more factors (e.g.,

password + hardware token + biometric).

3. Explain at least five good password-security practices. 1. Use long, unique passphrases (preferably 12+ characters) rather than short complex

strings. 2. Never reuse passwords across different sites or accounts. 3. Store credentials exclusively in a reputable password manager; never

write them down or keep them in plaintext files. 4. Enable multi-factor authentication on every account that supports it. 5. Change a password

immediately after any suspected compromise or data-breach notification, and monitor accounts for unusual activity.

4. Explain Password Hashing, Salt, and why plaintext password storage is dangerous. Password Hashing is a one-way cryptographic function

that converts a password into a fixed-length digest that cannot be reversed to recover the original password. A Salt is a unique, randomly

generated value concatenated with the password before hashing; it ensures that identical passwords produce different hashes and defeats

pre-computed rainbow-table attacks. Storing passwords in plaintext is extremely dangerous because a single database breach immediately

exposes every user’s credential, enabling credential-stuffing attacks against other services and complete account takeover with no additional

effort by the attacker.

5. Explain what a web session is and why applications use session identifiers. A web session is a temporary, server-side record of a user's

authenticated state that persists across multiple HTTP requests. Because HTTP is stateless, the server cannot otherwise know that successive

requests belong to the same logged-in user. Applications therefore issue a session identifier (commonly a high-entropy cookie or token) after

successful authentication; the client returns this identifier with each subsequent request so the server can look up the corresponding session

data and treat the user as authenticated without requiring re-login on every page.

6. Explain the purpose of Secure, HttpOnly and SameSite cookie attributes. Secure: Instructs the browser to transmit the cookie only over

encrypted HTTPS connections, preventing interception on plain HTTP. HttpOnly: Prevents client-side JavaScript from reading the cookie,

mitigating theft via Cross-Site Scripting (XSS). SameSite: Restricts when the cookie is sent on cross-site requests (Strict / Lax / None), thereby

reducing the risk of Cross-Site Request Forgery (CSRF) attacks. Together these three flags harden session cookies against the most common

web threats.

7. Explain Idle Timeout, Absolute Timeout and Session Invalidation on Logout. Idle Timeout: The session is automatically destroyed after a

defined period of user inactivity (e.g., 15–30 minutes). Absolute Timeout: The session is forcibly terminated after a hard maximum lifetime (e.g.,

8 hours) regardless of activity. Session Invalidation on Logout: When the user explicitly logs out, the server immediately deletes or marks the

session record as invalid so the same session identifier can no longer be reused, even if an attacker later obtains the cookie.

Jiwosh Shrestha — Day 33 MFA & Session Security

LAB ASSIGNMENT REPORT

DAY 33

Course: Web Security & Penetration Testing | Day 33 Practical Evidence

Jiwosh Shrestha

PRACTICAL LAB — MFA BYPASS OBSERVATION

(2 MARKS)

8. Briefly explain what security issue was demonstrated in the Day 33 MFA training lab and how developers can prevent it. The lab

demonstrated a classic “2FA simple bypass” vulnerability: after submitting valid first-factor credentials the application issued a session cookie

and allowed direct navigation to authenticated pages without ever verifying that the second factor (OTP) had been successfully completed. An

attacker could therefore skip the MFA challenge entirely by requesting the post-login endpoint. Developers must enforce server-side state that

records successful completion of every required authentication step, issue only short-lived intermediate tokens until MFA succeeds, and reject

any request that attempts to reach protected resources before the full MFA sequence is finished.

PRACTICAL EVIDENCE & WORKFLOW SCREENSHOTS

Identified Authentication Flow Observations:





